ALHAZ MOCKBUL HOSSAIN COLLEGE

Project Book

StayEase – Online Reservation and Booking System

Course Name: Software Engineering LAB | **Course Code: 530220**

Submitted To	SHARMIN AKTER JUI Lecturer, CSE Department Alhaz Mockbul Hossain College
Submitted By	MD. NAZMUL HOQUE
Roll	2461240
Reg	21502006239
Department	Department of Computer Science & Engineering (CSE)
Date	_____

_____

Signature

Table of Contents

Chapter 1: Introduction **2**

1.1 Project Overview 2

1.2 Problem Statement 2

1.3 Objectives 2

1.4 Key Features 2

Chapter 2: Methodology **3**

2.1 Development Model 3

2.2 Tools and Technologies 3

2.3 System Architecture 3

2.4 Double-Booking Prevention and Testing 4

Chapter 3: Design **5**

3.1 Flow Chart 5

3.2 E-R Diagram 6

3.3 Data Flow Diagram (DFD) 7

Chapter 4: Implementation **8**

4.1 Project Structure 8

4.2 Important Source Code 8

4.3 Screenshots 10

4.4 Conclusion 11

CHAPTER 1

Introduction

1.1 Project Overview

StayEase is a web-based **Online Reservation and Booking System** for a boutique hotel in Dhanmondi, Dhaka. Guests can browse rooms, check live availability and the total price for their dates, create an account and book a room online. Hotel staff use a separate admin panel to manage rooms, bookings, payments, customers and reviews. The system is built with **Next.js 16**, **TypeScript** and **MongoDB** and is live at **stayease-murex.vercel.app**.

1.2 Problem Statement

Many small hotels in Bangladesh still take reservations by phone and write them in a register. This causes:

- **Double booking** of the same room for overlapping dates.
- **No real-time availability** or clear pricing for guests.
- **Poor record keeping** and a high manual workload for staff.

1.3 Objectives

- Let customers search availability by date and book rooms online.
- Prevent double booking automatically using database transactions.
- Provide secure login with separate *Customer* and *Administrator* roles.
- Give the hotel one dashboard to manage rooms, bookings, payments, users and reviews.

1.4 Key Features

Table 1.1: Main features by user role

Customer	Administrator
Search rooms by dates and guests; see rooms left and total price	Dashboard: bookings, occupancy, revenue, today's arrivals and departures
Book a room and receive a reference code (e.g. SE-7K3P9Q)	Confirm, cancel or complete bookings and record payment
View and cancel bookings; review a completed stay	Manage room types, rooms, customers and reviews

CHAPTER 2

Methodology

2.1 Development Model

The project followed an **incremental development model**. The system was built in seven increments — setup, authentication, room catalogue, availability & booking, admin management, reviews & polish, and testing & deployment. Each increment was planned, built, tested and reviewed before the next one started.

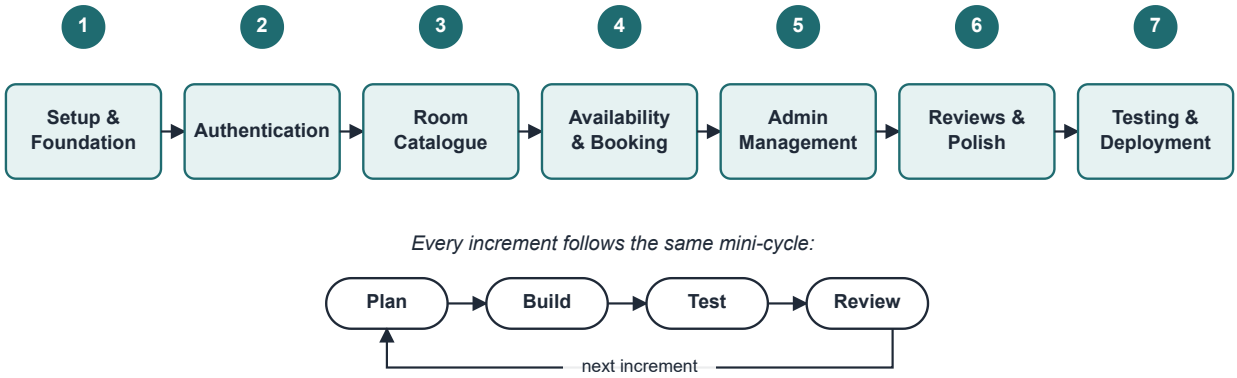


Figure 2.1: Incremental development model used for StayEase

2.2 Tools and Technologies

Table 2.1: Technology stack

Layer	Technology
Frontend & backend	Next.js 16 (App Router), React 19, TypeScript
User interface	Tailwind CSS 4, shadcn/ui, Lucide icons
Database	MongoDB Atlas with Mongoose 9
Validation & security	Zod, React Hook Form, JWT (HTTP-only cookie), bcrypt password hashing
Testing & deployment	Vitest, Git & GitHub, Vercel

2.3 System Architecture

StayEase uses a **three-tier architecture**: the browser shows the user interface, the Next.js server on Vercel runs the pages and business rules, and MongoDB Atlas stores the data.

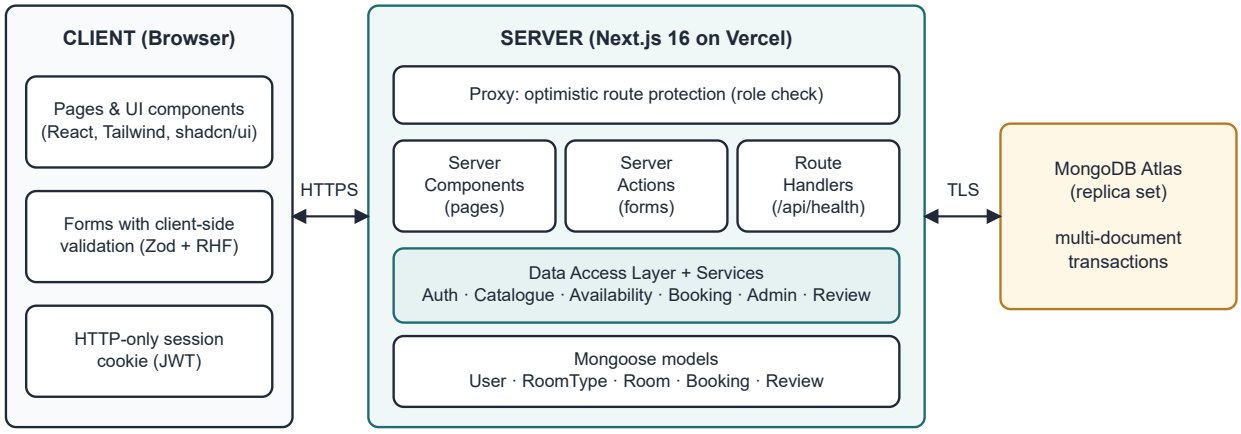


Figure 2.2: Three-tier architecture of StayEase

2.4 Double-Booking Prevention and Testing

Every booking is saved inside a **MongoDB transaction** that first locks a free room. If two customers book the same room at the same moment, only one succeeds; the other is moved to the next free room or told that no rooms are left. The system was checked with **159 automated tests** (all passed) — including twelve simultaneous booking requests for two rooms, which produced exactly two bookings — and by manual testing on desktop and mobile.

CHAPTER 3

Design

3.1 Flow Chart

Figure 3.1 shows how a customer searches, logs in and books a room. The final availability check is done inside a database transaction just before the booking is saved.

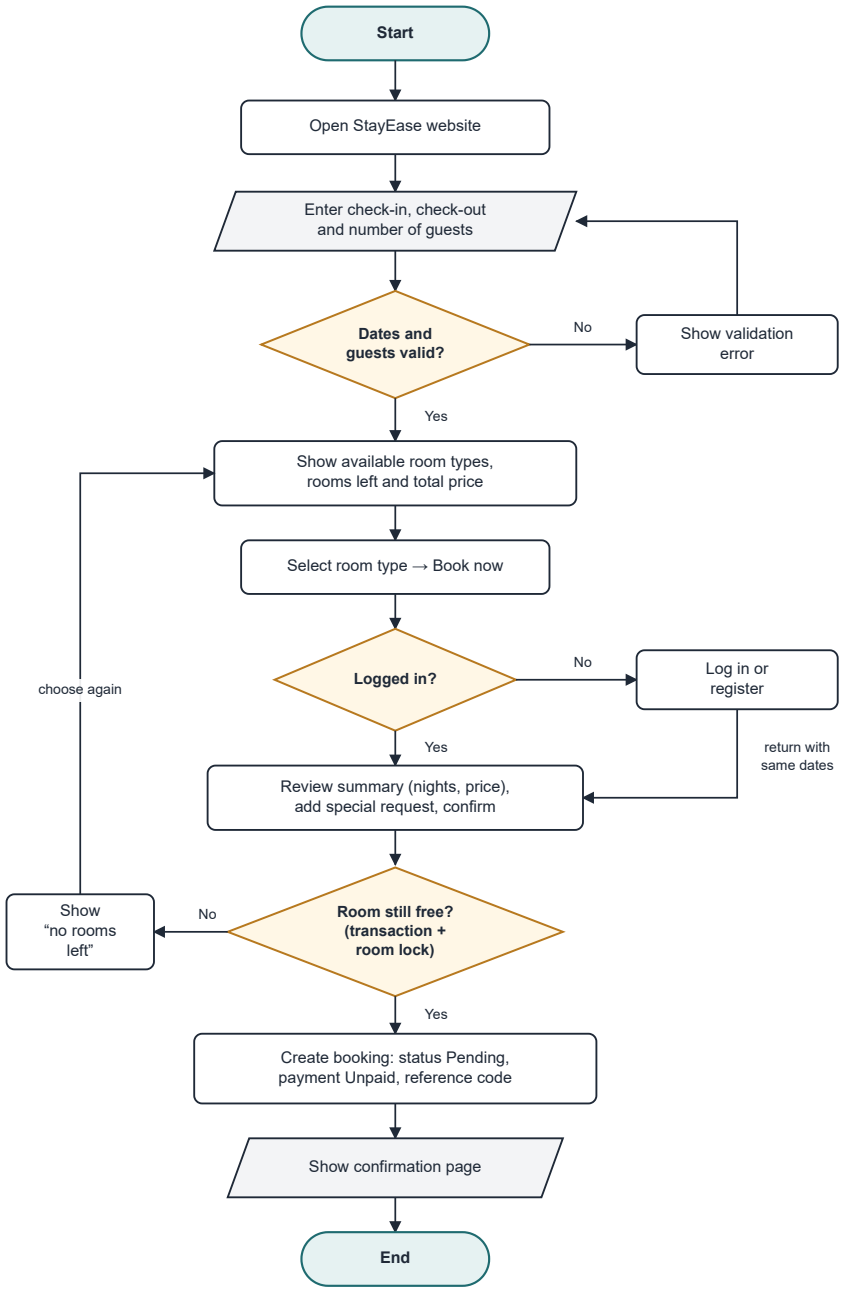


Figure 3.1: Flow chart of the customer booking process

3.2 E-R Diagram

A **USER** makes many **BOOKING**s; each booking is assigned one **ROOM**, and each room belongs to a **ROOM_TYPE**. A completed booking can have one **REVIEW**.

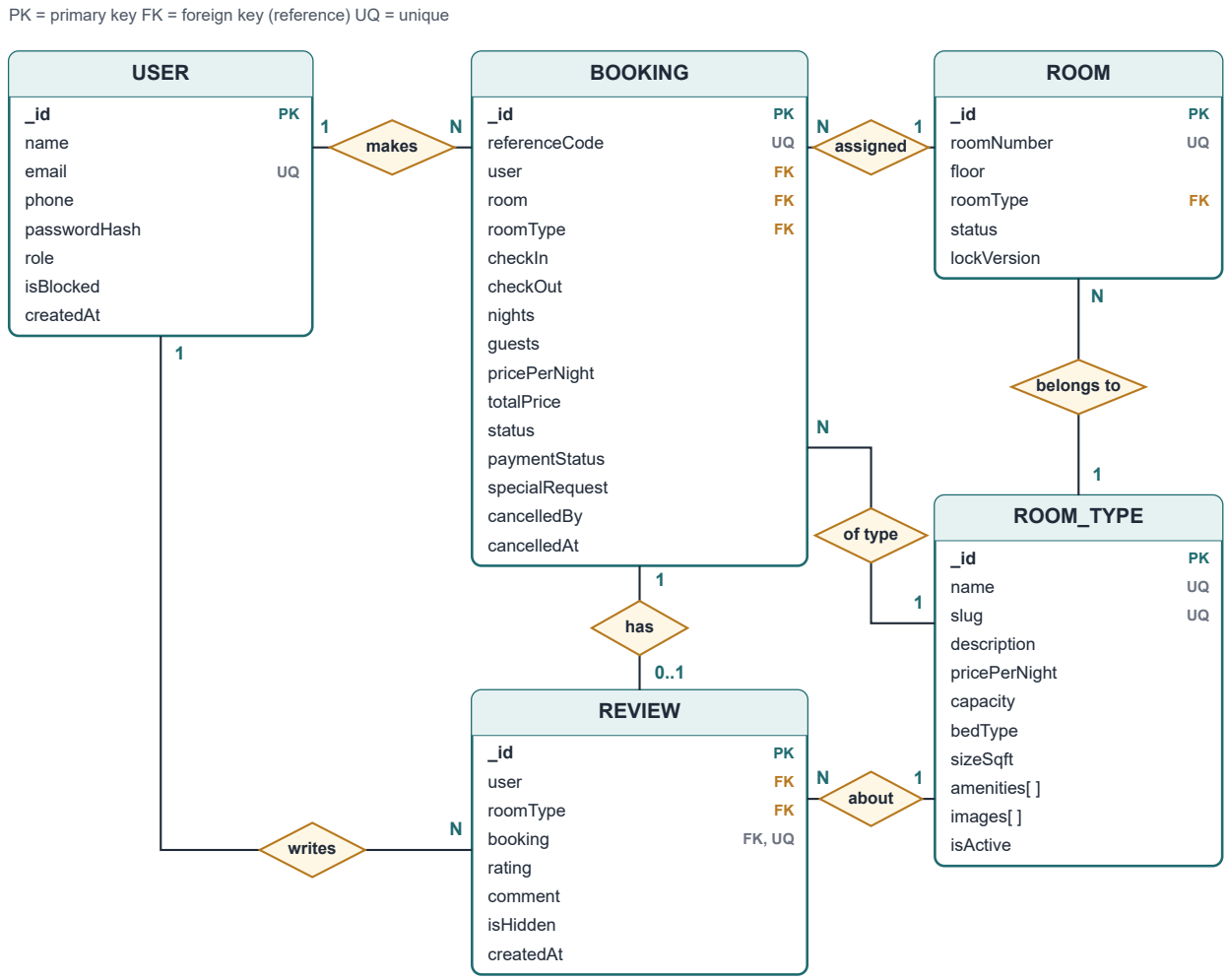


Figure 3.2: Entity-relationship diagram of StayEase

3.3 Data Flow Diagram (DFD)

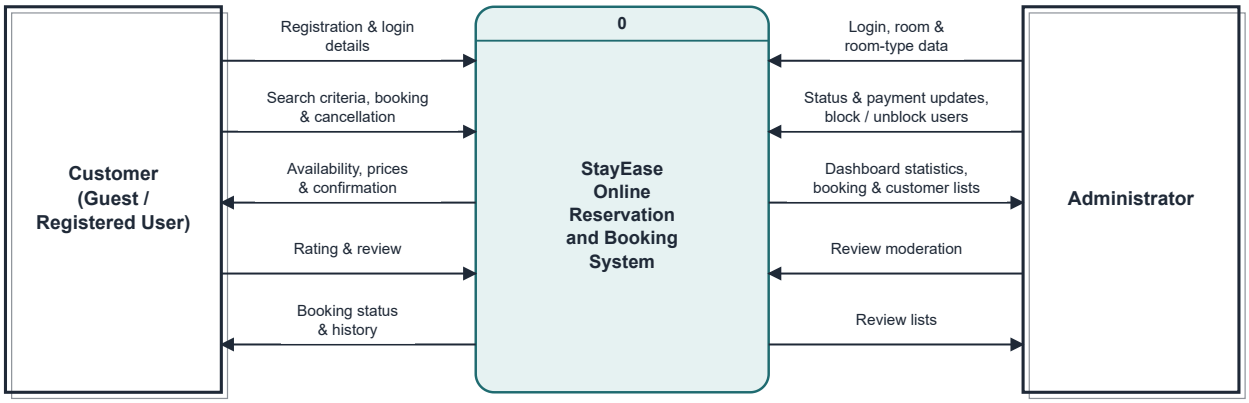


Figure 3.3: DFD Level 0 (context diagram)

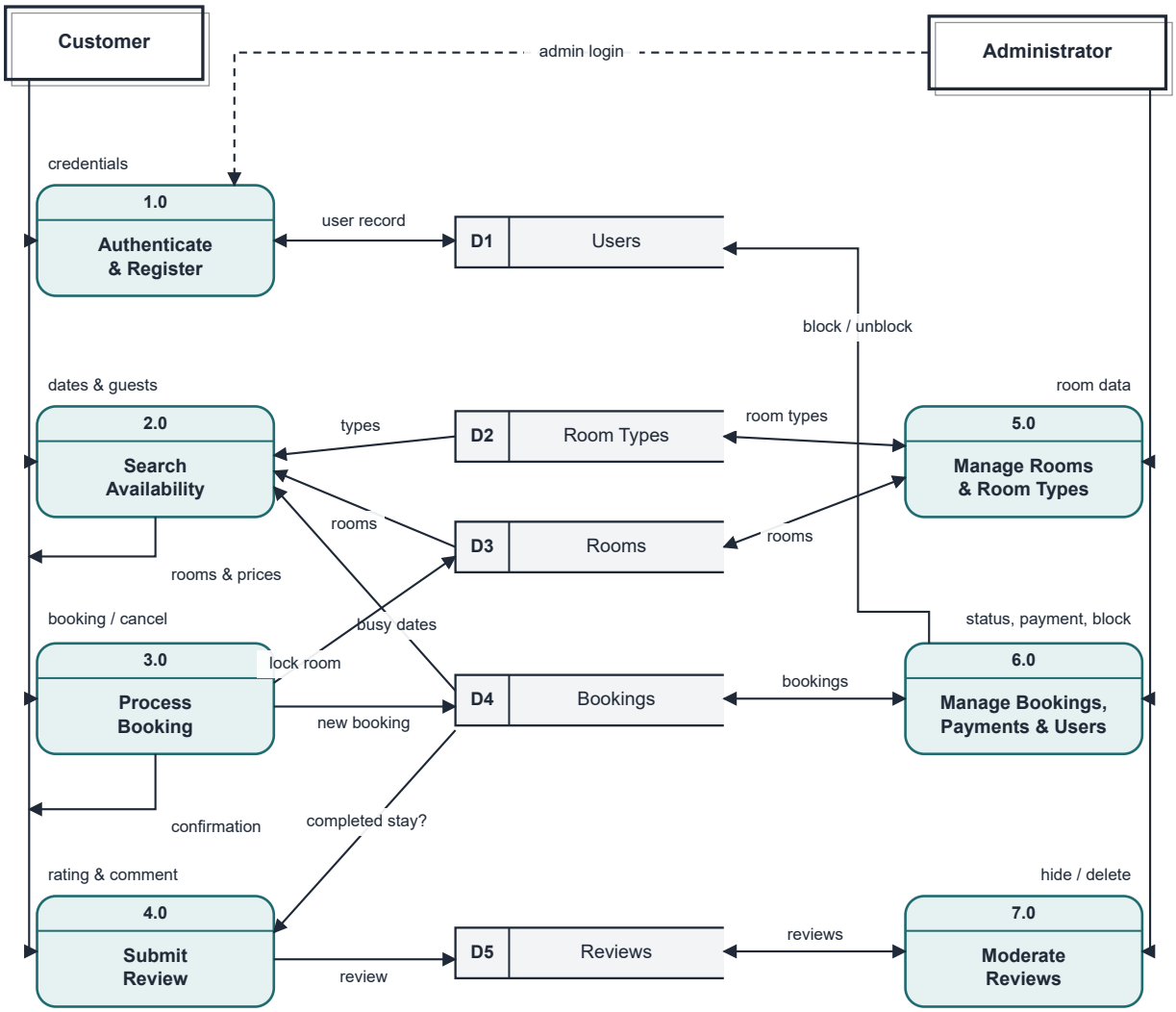


Figure 3.4: DFD Level 1

CHAPTER 4

Implementation

4.1 Project Structure

```
stayease/
├── src/
│   ├── app/          Pages: home, rooms, search, booking, login, admin
│   ├── actions/      Server actions (form submissions)
│   ├── components/   UI components
│   ├── lib/          Database connection, dates, authentication
│   ├── models/       User, RoomType, Room, Booking, Review
│   ├── services/     Business logic: availability, booking, admin
│   └── validations/  Zod schemas shared by forms and server
└── tests/            Automated tests (Vitest)
```

4.2 Important Source Code

Listing 4.1 is the availability rule: a booking blocks a room only if it is pending or confirmed and its dates overlap the requested stay.

Listing 4.1: Overlap rule for availability — src/services/availability.service.ts, lines 24–30

```
export function overlappingActiveBookingFilter(range: StayRange) {
  return {
    status: { $in: ACTIVE_BOOKING_STATUSES },
    checkIn: { $lt: parseDateString(range.checkOut) },
    checkOut: { $gt: parseDateString(range.checkIn) },
  };
}
```

Listing 4.2 creates a booking inside a transaction. It locks the first free room by increasing its `lockVersion` and then saves the booking with a server-calculated price, so two bookings can never be stored for the same room and dates.

Listing 4.2: Booking transaction with room lock — src/services/booking.service.ts, lines 130–172

```
const bookingId = await withTransaction(async (session) => {
  const roomType = await RoomType.findOne({ slug: params.roomTypeSlug, isActive: true })
    .session(session)
    .lean();
  if (!roomType) throw new BookingFailure("room_type_unavailable");
  if (params.guests > roomType.capacity) throw new BookingFailure("too_many_guests");

  const freeRooms = await findFreeRooms(roomType._id, range, session);

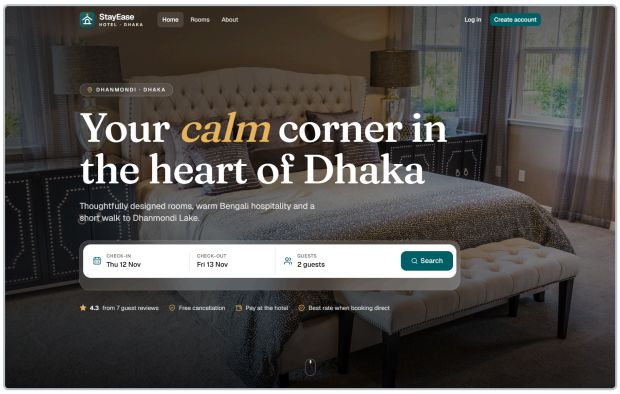
  for (const candidate of freeRooms) {
    // Room-level lock (§12.5). Matches only if the room is still available.
```

```
    const lock = await Room.updateOne(
      { _id: candidate._id, status: "available" },
      { $inc: { lockVersion: 1 } },
      { session },
    );
    if (lock.matchedCount === 0) continue;

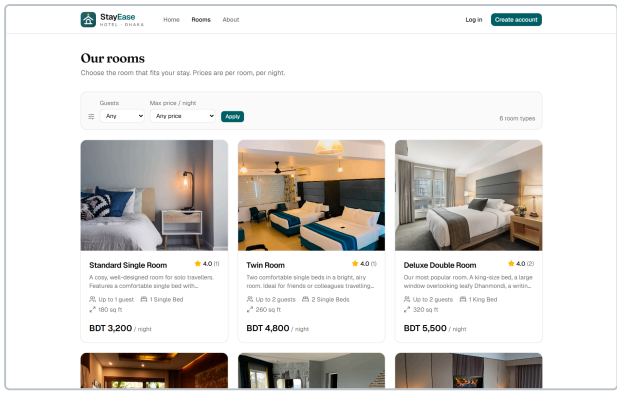
    const [booking] = await Booking.create(
      [
        {
          referenceCode: generateReferenceCode(),
          user: params.userId,
          room: candidate._id,
          roomType: roomType._id,
          checkIn: parseDateString(params.checkIn),
          checkOut: parseDateString(params.checkOut),
          nights,
          guests: params.guests,
          pricePerNight: roomType.pricePerNight, // snapshot (FR-BOOK-07)
          totalPrice: nights * roomType.pricePerNight, // server-side (FR-BOOK-03)
          status: "pending",
          paymentStatus: "unpaid",
          specialRequest: params.specialRequest,
        },
      ],
      { session },
    );
    return booking._id;
  }

  throw new BookingFailure("no_rooms_available");
});
```

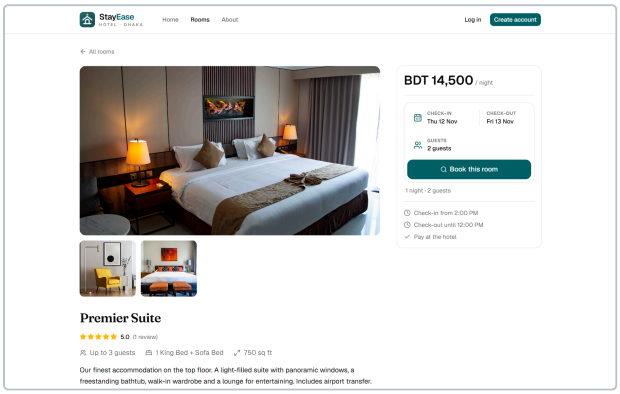
4.3 Screenshots



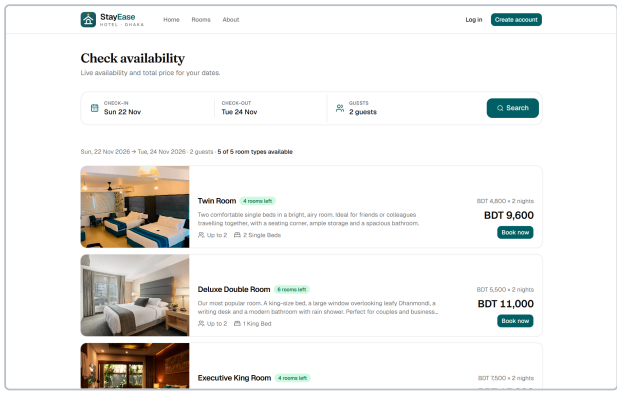
(a) Home page



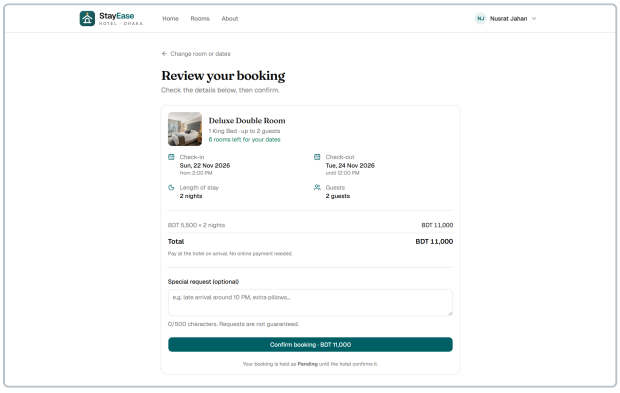
(b) Room list



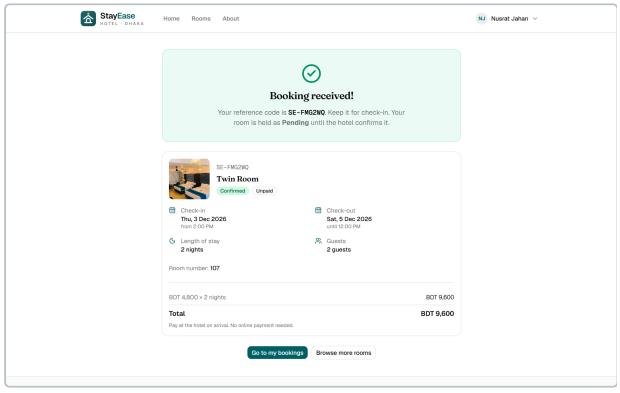
(c) Room details



(d) Availability search

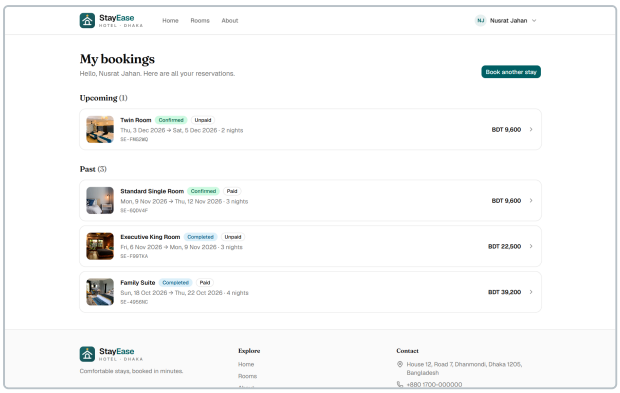


(e) Booking review

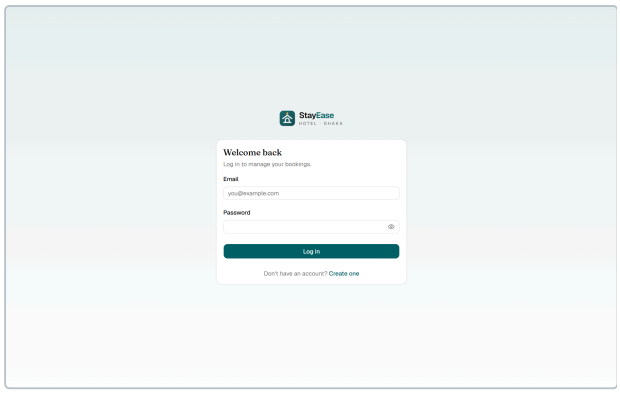


(f) Booking confirmation

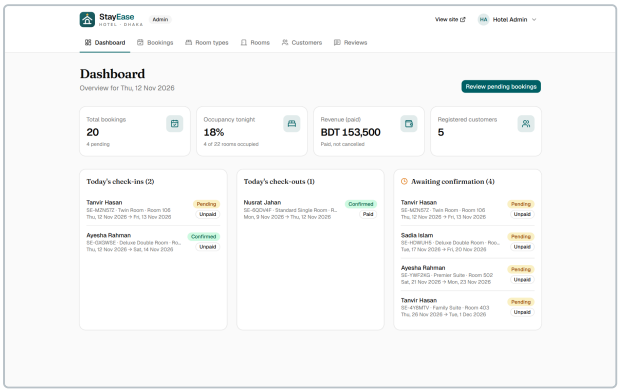
Figure 4.1: Customer side of StayEase



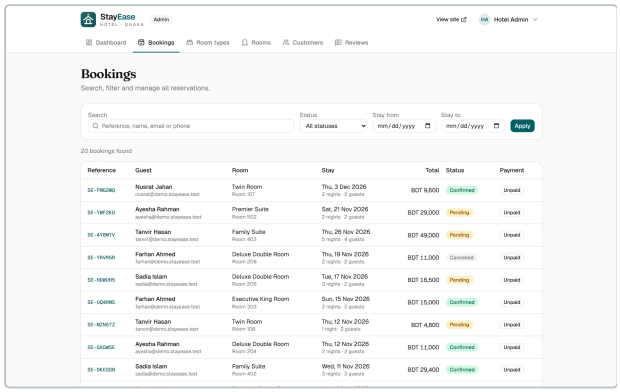
(a) My bookings



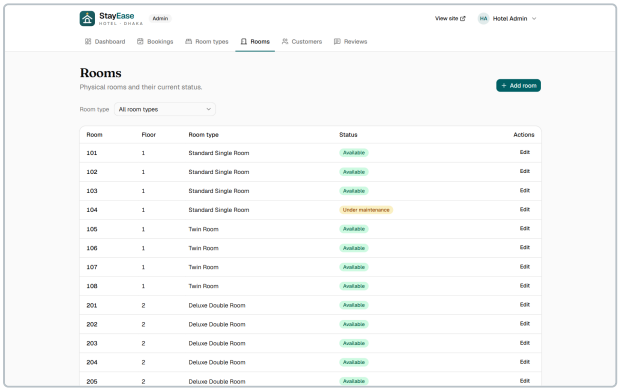
(b) Login



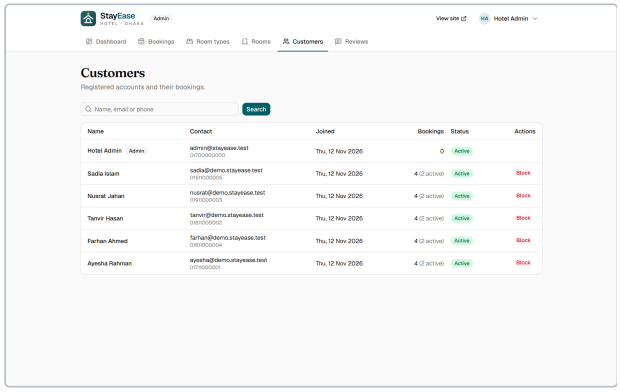
(c) Admin dashboard



(d) Booking management



(e) Room management



(f) Customer management

Figure 4.2: Customer account and administrator panel

4.4 Conclusion

StayEase meets its objectives: guests can find and book a room online, the hotel manages all reservations from one dashboard, and double bookings are prevented by design and proven by automated tests. Future work includes online payment, e-mail notifications and seasonal pricing.